

DOMANDE SECONDO ESONERO 2021

1. ARCHITETTURE PARALLELE: quali tra le seguenti affermazioni NON sono vere se si considerano le architetture UMA basate su bus, crossbar switch e rete omega
 - ✓ La rete omega è non bloccante
 - Il crossbar switch è bloccante
 - le interconnessioni richieste in una rete omega sono superiori a quelle di un crossbar switch
 - ✓ il bus può divenire il collo di bottiglia quindi sono possibili fino a 100 CPU
 - ✓ in una rete omega con n CPU e n memorie il numero di switch cresce come $O(n \log n)$

2. ASSEMBLY ARM - Supponendo che il registro R0 contenga il valore E8 con quali istruzioni si può inserire in R1 il valore E800E8?
 - ✓ MOV R1, #0x00E800E8
#questa istruzione assegna il valore E800E8 direttamente al registro R1
 - ✓ PKHBT R1, R0, R0, LSL #16
#questa istruzione fa un pack high byte top, ovvero una concatenazione del byte alto di R0 con il byte basso di R0 spostato di 16 bit a sinistra, il risultato viene inserito in R1
 - ✓ ADD R1, R0, R0, LSL #16
questa istruzione esegue un'operazione di somma tra R0 e R0 spostato di 16 bit a sinistra, il risultato viene inserito in R1
 - UXTB16 R1, R2
 - ADD R1, R0, R0, LSL #8
 - LDR R1, = 0x00E800E8

3. LIVELLO DI MICROARCHITETTURA: quali affermazioni NON sono vere nell'architettura Mic-1?
 - ✓ La ALU non è in grado di eseguire operazioni di sottrazione tra gli operandi
in realtà, la ALU è in grado di eseguire diverse operazioni aritmetiche, incluse le operazioni di sottrazione
 - ✓ Le microistruzioni sono scritte in sequenza
in realtà, le microistruzioni vengono eseguite in parallelo
 - La microistruzione contiene anche i segnali destinati alla ALU
 - ✓ il decoder ha lo scopo di abilitare la scrittura nei registri di memoria
in realtà, il decoder ha lo scopo di decodificare le istruzioni e generare i segnali di controllo necessari per eseguirle
 - Esiste un registro OPC per contenere l'ultima istruzione valutata

4. ARCHITETTURE PARALLELE: quali sono le limitazioni delle chiamate a procedura remota?

- ✓ Non si possono utilizzare riferimenti di memoria come parametri
invece, i parametri devono essere passati per valore
- ✓ Non è possibile utilizzare le variabili globali
invece, le variabili globali devono essere passate come parametri o utilizzate attraverso un meccanismo di indirizzamento condiviso
- Non è possibile utilizzare parametri con tipi primitivi (es. interi)
- ✓ Non si può utilizzare il polimorfismo sui dati
invece, è necessario utilizzare un meccanismo di indirizzamento condiviso per accedere ai dati polimorfici
- ✓ Non è possibile utilizzare variabili locali
invece, è necessario utilizzare un meccanismo di indirizzamento condiviso per accedere alle variabili locali

Nota: l'affermazione "Non è possibile utilizzare parametri con tipi primitivi (es. interi)" non è vera in generale, poiché è possibile utilizzare parametri di qualsiasi tipo con le chiamate a procedura remota. Tuttavia, può essere vero che alcuni sistemi di chiamata a procedura remota possono avere limitazioni sui tipi di parametri che possono essere utilizzati.

5. TEOREMA DI BOHM-JACOPINI: quali affermazioni NON sono vere

- ✓ Sequenza, Iterazione e selezione costituiscono i concetti base dei DNS
in realtà, i concetti base dei DNS sono la sequenza, l'iterazione e la selezione, non la sezione
- La presenza di istruzioni di uscita incondizionata (es. BREAK) costituiscono il fondamento per la destrutturazione del codice
- Si può dimostrare che la presenza di annidamenti complessi in un algoritmo risolutivo possono invalidare la traduzione in un equivalente algoritmo strutturato
- ✓ Un algoritmo non strutturato è sempre possibile tradurlo in qualsiasi linguaggio di programmazione
in realtà, alcuni algoritmi non strutturati potrebbero non essere facilmente traducibili in linguaggi di programmazione strutturati
- ✓ I DNS permettono di scrivere algoritmi rispettando i concetti della programmazione strutturata
in realtà, i DNS consentono di scrivere algoritmi utilizzando solo la sequenza, l'iterazione e la selezione, mentre la programmazione strutturata introduce ulteriori concetti come le funzioni e la gestione delle eccezioni)

6. SISTEMI OPERATIVI: Nell'ambito dei mutex se l'istruzione di Test & Set Lock dell'ARM è TST ed il codice assembly per realizzare il lock del mutex il seguente:

```
mutex_lock: TST R0, MUTEX
             CMP R0, #0
             BEQ fine
             <istruzione mancante>
             B mutex_lock
fine:        BX chiamante
```

Qual è l'istruzione mancante nel linguaggio ARM?

- BL thread_yield
- NOP
- BX thread_yield
- ✓ MOV MUTEX, #0
questa istruzione imposta il mutex a 0, ovvero lo acquisisce
- CALL thread_yield

Quindi: Il codice completo per il lock del mutex dovrebbe quindi essere:

```
mutex_lock: TST R0, MUTEX
CMP R0, #0
BEQ fine
MOV MUTEX, #0
B mutex_lock
fine: BX chiamante
```

7. SISTEMI OPERATIVI - Nell'ambito degli algoritmi di sostituzione delle pagine quali affermazioni non sono vere?

- ✓ Tutte le approssimazioni dell'LRU possono lavorare solo con politiche di sostituzione delle pagine globali.
in realtà, alcune approssimazioni dell'LRU possono anche lavorare con politiche di sostituzione delle pagine locali
- WSClock può anche lavorare con politiche di sostituzione delle pagine locali
- ✓ Working Set può lavorare esclusivamente con politiche di sostituzione delle pagine locali
in realtà, Working Set può lavorare sia con politiche di sostituzione delle pagine locali che con politiche di sostituzione delle pagine globali
- LRU può lavorare anche con politiche di sostituzione delle pagine globali.
- ✓ FIFO non può lavorare con politiche di sostituzione delle pagine locali
in realtà, FIFO può lavorare sia con politiche di sostituzione delle pagine locali che con politiche di sostituzione delle pagine globali

8. SISTEMI OPERATIVI: quali condizioni non sono necessarie per evitare le corse critiche?

- ✓ Nessun processo può eccedere il suo quantum di tempo
questa condizione può aiutare a evitare le corse critiche, ma non è necessaria)
- Nessun processo in esecuzione al di fuori della sua regione critica può bloccare altri processi
- Due processi non possono rimanere all'interno delle loro regioni critiche allo stesso tempo
- ✓ Se un processo è più veloce degli altri può mantenere assegnate le risorse
questa condizione può aiutare a evitare le corse critiche, ma non è necessaria
- Nessun processo deve restare in attesa infinita per poter entrare nella sua regione critica

9. SISTEMI OPERATIVI: Nell'ambito della gestione dei processi quali affermazioni NON sono vere?

- ✓ Nel caso ci siano più CPU occorre proteggere il semaforo con istruzioni TSL (o XCHG)
in realtà, non è necessario proteggere il semaforo con istruzioni TSL o XCHG in presenza di più CPU, poiché queste istruzioni sono già atomiche
- La soluzione di Peterson non richiede busy waiting
- Tutte le operazioni in un semaforo sono atomiche
- ✓ L'istruzione TSL fa sprecare del tempo di CPU perché richiede busy waiting
in realtà, l'istruzione TSL non richiede busy waiting, poiché è atomicamente eseguita
- La soluzione di Peterson non risente del problema dell'inversione delle priorità

10. SISTEMI OPERATIVI: Nella gestione dei dispositivi di I/O quali non sono svantaggi del port-mapped I/O

- ✓ Il sistema operativo non riesce ad assegnare in modo semplice e dinamico i dispositivi a processi utente
- ✓ È necessario un meccanismo di protezione speciale per controllare lo svolgimento delle operazioni di I/O da parte dei processi utente
- ✓ Per leggere e scrivere i registri di controllo del dispositivo, sono necessarie istruzioni dedicate assembly: IN e OUT
- I registri di controllo del device sono visti come variabili in memoria
- ✓ I driver di controllo dei dispositivi possono essere scritti utilizzando solo il linguaggio C

11. SISTEMI OPERATIVI - Descrivere l'algoritmo ottimo di sostituzione delle pagine indicando la sequenza partendo dall'alto verso il basso

- accade un page-fault
- per ogni pagina caricata si calcola il numero di istruzioni che sarà eseguito prima che quella pagina sia referenziata
- si etichetta ogni pagina caricata con il numero di istruzioni che sarà eseguito prima che quella pagina sia referenziata
- si scorrono tutte le pagine ricercando quella con etichetta massima
- si sostituisce la pagina che ha l'etichetta più alta

Risposta: L'algoritmo ottimo di sostituzione delle pagine (noto anche come Optimal Page Replacement Algorithm) è un algoritmo teorico che viene utilizzato per prevedere quale pagina dovrebbe essere sostituita in un sistema di paging a memoria virtuale. L'idea alla base di questo algoritmo è che la pagina che sarà utilizzata ultima in futuro è quella che dovrebbe essere mantenuta nella memoria.

La sequenza di operazioni per l'algoritmo ottimo di sostituzione delle pagine è la seguente:

1. Si verifica un page-fault, ovvero si tenta di accedere a una pagina che non è presente nella memoria.
2. Si calcola per ogni pagina caricata il numero di istruzioni che sarà eseguito prima che quella pagina sia referenziata di nuovo.
3. Si etichetta ogni pagina caricata con il numero di istruzioni calcolato.
4. Si scorrono tutte le pagine in memoria e si cerca quella con l'etichetta più alta.
5. Si sostituisce la pagina con l'etichetta più alta con la nuova pagina che si vuole caricare in memoria.

L'algoritmo ottimo di sostituzione delle pagine è considerato il riferimento per la valutazione delle prestazioni degli altri algoritmi di sostituzione delle pagine. Tuttavia, poiché richiede di conoscere in anticipo l'intera sequenza di riferimenti alle pagine, non può essere implementato in modo pratico.